Lecture 3: Miscellaneous Pythonic Programming

Instructor: Jordan Schwartz

Slides adapted from Tony Liu

Logistics

- HW1 is due **two** weeks from today. (I accidentally said two weeks last week. I was wrong)
  - You will need content from next week to do the hw (testing)
- There is an Ed post to ask any follow up questions about lecture if we don't get to them

Oops, syllabus wasn't updated

The late policy that we meant to have 😅:

In addition to the grace period, every student gets 5 late days that they may use at any point in the semester. You may use up to 1 late day on any one assignment. Late days are automatically applied for any submission submitted after the grace period deadline. This means the latest any assignment may be submitted is 48 hours after 30 minutes prior to the start of class the day the assignment is originally due.

Side note: you can always reach out **before the deadline** if you need more time to complete the assignment

Introductions - as quick as possible

Name

Best type of candy and worst type of candy

Week 3

- From last week: positional and keyword args

- Object Oriented Programming in Python
- Iterators and Generators
- Higher Order Functions
- File I/O

Positional and Keyword Arguments

[code demo]

🔑: function arguments can be referred to via position, or by keyword
🔑: can even provide optional arguments and default values

Positional unpacking

* explicitly unpacks a sequence in a function call **or** definition

[code demo]

Keyword unpacking

** explicitly unpacks a dictionary in a function call **or** definition

[code demo]

Check in

```
d = {'a': [2, 3], 'b': [3, 4], 'c': [4, 5]}
```

M1

```
def foo(bar, **barbar):
    for k, v in barbar.items():
        print(bar[k][0] + v[1])
foo(d, **d)
```

M2

```
def foo2(bar, a, b, c):
    for k, v in bar.items():
        if a == k:
            print(a[0] + v[1])
        elif b == k:
            print(b[0] + v[1])
        else:
            print(c[0] + v[1])
foo2(d, **d)
```

M3

```
def foo4(**bar, barbar):
    for k, v in bar:
        print(barbar[k][0] + v[1])
foo4(d, **d)
```

A
5
7
9

B
7
8
9

C Error

Objects and OOP

Objects

- So far some objects in Python we've seen are:
  - Lists
  - Tuples
  - Sets
  - Dictionaries
- What are objects usually used for?
  - Objects are typically used to store some data
- We can define our own objects to store data in our own format
- We create objects from a blueprint: a class

Examples of Objects Created from a Class

Classes

- Capitalize the class name
- `self` gets passed in implicitly for the object before the .

[code demo]

```
def Dog():
    # class attributes
    num_legs = 4

    # constructor, self is like "this" in Java
    def __init__(self, age, name):
        # instance attributes
        self.age = age
        self.name = name

        self.owners = []

    # instance method, self is always the first argument
    def age_up(self):
        self.age += 1
```

Check in - L11

Write an `add_owner` method that takes in a string as the owner's name and adds that string to the appropriate instance attribute.

If you finish early, add a `num_owners` method that returns the number of owners a dog instance has.

If you still have time, write a `num_owners` method that prints out all the owners using string formatting..

```
def Dog():
    # class attributes
    num_legs = 4

    # constructor, self is like "this" in Java
    def __init__(self, age, name):
        # instance attributes
        self.age = age
        self.name = name

        self.owners = []

    # instance method, self is always the first
argument
    def age_up(self):
        self.age += 1
```

Inheritance

[code demo]

- attributes referenced in the derived class and not found are searched for in the base/super class
- derived/sub classes may override base/super class methods, or call base/super class methods with super().method()

Attribute Scope

- the Python interpreter resolves attribute names in the following order
  - 1. Local scope: code block of current function
  - 2. Enclosing scope: code block of nested function (rare)
  - 3. Global scope: top-level scope in your Python program/file
  - 4. Built-in scope: special scope containing all built-ins

[code demo]

Magic/dunder methods

- “dunder” == “double underscore”
- Methods that can be overridden to emulate built-in Python types
- https://docs.python.org/3/reference/datamodel.html#specialnames

[code demo]

Check in: C12

What if we wanted to display a dog's info instead of the ugly pointer?

Write a new method for the dog class that will display the age and name prettily when we call print.

Hint: the repr method prints out **computer** readable representation of an object and the str method prints out the **human** readable representation of an object. Which is used when we call print

If you finish early, write a method for the computer representation that displays "`Object [dog's name]`"

Iterables and Iterators

Iterables

- an iterable is an object that can return its elements one at a time
- any object that implements the `__iter__` magic method is an iterable
- (check the docs for fine print)

[code demo]

Iterators

- Are a type of iterable (hierarchical relationship)
- `__iter__` returns an iterator object
  - the iterator can be accessed by `iter(obj)`
- Return their elements one at a time using the `next()` method (implemented as `__next__`)
- when the iterator runs out of elements, a `StopIteration` exception is raised
- A single iterator object can only be gone through exactly 1 time
- Maintains some internal state

[code demo]

Generators

- Are a type of iterator!
  - What does this mean? What method(s) do they implement?
- Defined similarly to a function, rather than using a dunder method to create it from an iterable
- `yield` returns data sequentially **in a sequence**
- automatically raises StopIteration
- automatically implements `__next__` and `__iter__`

[code demo]

Example: recursive generation with subsequences

given"abc", yield "", "a", "b", "c", "ab", "ac", "bc", "abc"

Anonymous Functions

Lambda functions

- Are anonymous functions → one line functions defined **without** a name

```
lambda <parameter list>: expression
==
def no_name(<parameter list>):
    return expression
```

[code demo]

Higher Order Functions

What data type are functions?

```
def greeting():
    print("hello")
# without parens, functions are objects
# nothing is telling the function to evaluate
>>> type(greeting)
<class 'function'>
```

🔑: we can treat functions as objects. And what can we do with objects?

[code demo]

[code demo]

Filter, Map, Reduce (a touch of functional programming)

Filter	Map	Reduce

`filter(f, seq)`
is equivalent to:
`[elem for elem in seq if f(elem)]`
	`map(f, seq)`
	is equivalent to:
	`[f(elem) for elem in seq]`
		is a function that takes in an aggregation function and a sequence and repeatedly accumulates elements from the sequence using the aggregation function.
		`from functools import reduce`
		`reduce(f, seq)`
		is roughly equivalent to:
		`result = seq[0]`
		`for elem in seq[1:]:`
		`  result = f(result, elem)`

Check In

M4: `list(filter(lambda x: x % 2 == 0, [0, 1, 2, 3]))`
M5: `list(map(lambda y: y % 2 == 0, [0, 1, 2, 3]))`
M6: `reduce(lambda x, y: x + y, [[0], [1], [2], [3]])`

Match to -
A: `[True, False, True, False]`
B: `[0, 2]`
C: `[0, 1, 2, 3]`
D: Error

File I/O and context management

Basic I/O

- input(): read from terminal stdin, stop after newline
- print(str): display the str in terminal, stdout

File Objects

- File objects expose an API to an underlying file resource

- open() returns a file object, with 2 arguments: open(filename, mode)
- close(): destructor for file object

File modes

Character	Meaning
'r'	open for reading (default)
'w'	open for writing, overwriting if file already exists
'x'	open for exclusive creation, fail if exists
'a'	open for writing, appending to the end of the file if it exists
'b'	open in binary mode
't'	open in text mode (default)
'+'	open for updating (reading and writing)

File reading and writing

[code demo]

Context management: with keyword

- with keyword allows for resource management in enclosed context
  - For file I/O, easy management/cleanup of file objects
- tedious since it is a common code pattern, can be error prone

```
try:
    # ...perform file operations...
    f = open("python_zen.txt")
    for line in f:
        print(line.strip())
except:
    # ...handle exceptions...
    pass
finally:
    # don't forget to clean up file
object!
    f.close()
```

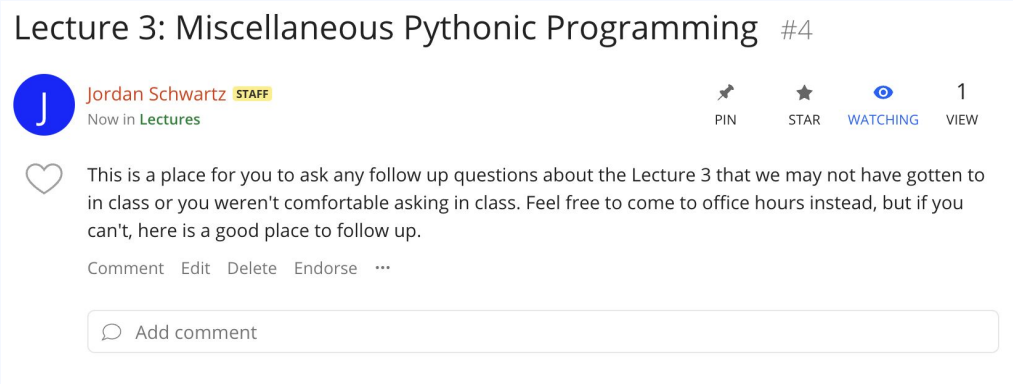
with Simplifies Things

```
# same as above, with context
management
with open("python_zen.txt") as f:
    for line in f:
        print(line.strip())
        #"line\n"
    #raise Exception
#print(f.closed)
```

Context Management

- a context manager defines two magic functions:
  - __enter__: called upon entering the with context
  - __exit__: called upon exiting the with context, even if an exception is thrown
- allows for boilerplate definition of clean-up actions and resource management
  - thread creation/deletion, database connection open/close...

Thanks and please post any follow up questions in the relevant Ed thread!



Thanks!

Do you have any questions?

youremail@freepik.com
+34 654 321 432
yourwebsite.com



Instructions for use

If you have a free account, in order to use this template, you must credit **Slidesgo** by keeping the **Thanks** slide. Please refer to the next slide to read the instructions for premium users.

As a Free user, you are allowed to:

- Modify this template.
- Use it for both personal and commercial projects.

You are not allowed to:

- Sublicense, sell or rent any of Slidesgo Content (or a modified version of Slidesgo Content).
- Distribute Slidesgo Content unless it has been expressly authorized by Slidesgo.
- Include Slidesgo Content in an online or offline database or file.
- Offer Slidesgo templates (or modified versions of Slidesgo templates) for download.
- Acquire the copyright of Slidesgo Content.

For more information about editing slides, please read our FAQs or visit our blog:
https://slidesgo.com/faqs and **https://slidesgo.com/slidesgo-school**